

Lab 10.1 - Pin ol-lab-01 & Prove It Took

KVM Virtualisation on Oracle Linux - Day 2, Module 10

Overview

In this lab you will capture ol-lab-01's current, unpinned CPU and memory placement as a baseline, then pin its vCPUs to specific physical cores on a single NUMA node and lock its memory to that same node. You'll finish by re-running the exact same baseline commands and comparing the before and after output directly - proof that the pinning genuinely took effect, not just that the commands ran without an error.

This lab assumes ol-lab-01 exists and is running, with 2 vCPUs and 2 GB of memory from Lab 3.1.

Note: If your host only exposes a single NUMA node (common on smaller or nested cloud instances - check with `numactl --hardware` in Step 1), the pinning commands still work exactly the same way and are worth doing correctly regardless. You simply won't see a dramatic node-to-node difference in the after state, since there's only one node available to begin with. The mechanics learned here transfer directly to any larger, multi-node production host.

Learning Objective

By the end of this lab, you will be able to:

- Read a host's NUMA topology and identify which physical cores belong to which node
- Capture a VM's baseline CPU and memory placement before making any change
- Pin a VM's vCPUs to specific physical cores on a single NUMA node
- Lock a VM's memory allocation to that same node
- Verify pinning genuinely took effect, rather than trusting that a command with no error means the job is done

Step by Step Guide

Prerequisite: [Connect to Your Lab VM](#)

Your OCI lab host was provisioned via the Terraform workflow covered earlier in the programme.

1. Connect to your lab host over SSH:

```
ssh opc@<your-lab-instance-ip>
```

Expected result: A shell prompt on your OCI lab instance.

2. Confirm ol-lab-01 exists and is running:

```
sudo virsh list --all
```

Expected result: ol-lab-01 listed with state running. If it's shut off, start it with sudo virsh start ol-lab-01 before continuing.

Step 1: Read the Host Topology

3. Check the host's socket, core, and NUMA node counts:

```
lscpu | grep -E "NUMA|Socket|Core|Thread"
```

4. Get the detailed picture - which CPUs belong to which node, how much memory each node has, and the distance between nodes:

```
numactl --hardware
```

Expected result: At least one NUMA node listed (node 0), showing its CPU list and memory size. If more than one node is present, note the CPU list for node 0 specifically - you'll pin ol-lab-01's vCPUs to cores from that list in Step 3.

Note: A distance of 10 in the node distance table means local access; higher numbers mean progressively more expensive remote access between nodes.

Step 2: Capture the Baseline

Before changing anything, record exactly how ol-lab-01's CPU and memory are currently placed, so Step 5 has something concrete to compare against.

5. Capture the VM's current memory distribution across NUMA nodes:

```
numastat -c qemu-system-x86
```

Expected result: ol-lab-01's QEMU process listed, with its memory shown split across however many nodes it currently happens to be using - likely more than one node if the host has several, since nothing has constrained it yet.

6. Capture which physical CPUs its vCPUs are currently allowed to run on:

```
sudo virsh vcpuinfo ol-lab-01
```

Expected result: Both vCPUs listed, each showing a CPU Affinity mask that spans every physical CPU on the host (typically shown as a long string of y characters) - confirmation nothing is pinned yet.

Note: Keep this terminal output visible or copy it somewhere - Step 5 asks you to compare against it directly.

Step 3: Pin vCPUs to Cores on a Single NUMA Node

Using the CPU list for node 0 from Step 1, pin each of ol-lab-01's two vCPUs to a distinct physical core within that node.

7. Pin vCPU 0 and vCPU 1 to two cores from node 0's CPU list - substitute the actual core numbers from your own numactl --hardware output:

```
sudo virsh vcpupin ol-lab-01 0 <core-a> --live --config
```

```
sudo virsh vcpupin ol-lab-01 1 <core-b> --live --config
```

Expected result: Both commands complete with no error. --live applies the pin immediately to the running VM; --config makes it persist across a reboot too.

Note: Choose two different cores for the two vCPUs, and avoid picking a pair that are actually hyperthread siblings of the same physical core if your topology has hyperthreading enabled - check lscpu's thread-per-core value from Step 1 first. Pinning both vCPUs onto siblings of the same core defeats much of the purpose of pinning.

8. Confirm the pin is in effect:

```
sudo virsh vcpuinfo ol-lab-01
```

Expected result: Each vCPU's CPU Affinity mask now shows y only at the single core it was pinned to, and CPU: shows the physical core it's currently running on.

Step 4: Lock Memory to That Node

Pinning vCPUs to the right cores accomplishes little if the VM's memory is still allocated wherever the host happens to find space. This step closes that gap.

9. Lock ol-lab-01's memory to node 0, in strict mode - the strongest guarantee available:

```
sudo virsh numatune ol-lab-01 --mode strict --nodeset 0 --live --config
```

Expected result: Command completes with no error. If node 0 doesn't have enough free memory for the VM, this fails outright rather than quietly allocating from another node - that's deliberate, since a memory pin that silently spills over defeats its own purpose.

10. Confirm the setting is applied:

```
sudo virsh numatune ol-lab-01
```

Expected result: numa_mode shows strict and numa_nodeset shows 0.

Step 5: Re-run Both Commands and Confirm the Before/After Difference

This is the step that actually proves the pinning worked, rather than just trusting that Steps 3 and 4 completed without an error.

11. Re-run the exact same memory distribution check from Step 2:

```
numastat -c qemu-system-x86
```

Expected result: ol-lab-01's memory now shown concentrated almost entirely on node 0, compared to the more spread-out baseline captured in Step 2.

12. Re-run the exact same vCPU affinity check from Step 2:

```
sudo virsh vcpuinfo ol-lab-01
```

Expected result: Each vCPU's CPU Affinity mask now shows a single pinned core, compared to the every-core mask captured in Step 2.

13. Place the Step 2 and Step 5 outputs side by side and confirm the difference directly - this comparison, not the absence of errors in Steps 3 and 4, is the actual evidence the lab set out to produce.

Outcomes

By the end of this lab, ol-lab-01's two vCPUs are pinned to specific physical cores on a single NUMA node, and its memory allocation is locked to that same node in strict mode. You captured a genuine before-and-after comparison using numastat and vcpuinfo, rather than relying on clean command output as proof - the same discipline that matters when applying this tuning to a real production workload, where the difference between configured and verified is exactly what separates a VM that merely runs from one that performs predictably under load.